Data Manipulation Language (DML)

Database Management Systems

INSERT INTO

- The INSERT INTO statement is used to insert new records in a table.

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

- Example:

```
INSERT INTO PERSONS(PersonID, FirstName, LastName, Address1, City)
Values(1,'asad', 'Kamal', 'House no 1', 'Johar Town','Lahore')
```

Multiple Values in single query

- Multiple rows can be added through single query

```
INSERT INTO PERSONS(PersonID, FirstName, LastName, Address1, City)
Values(1,'asad', 'Kamal', 'House no 1', 'Johar Town','Lahore'),
(2,'Muhammad', 'Mumtaaz', 'House no 2', 'Johar Town','Lahore'),
(3,'Bilal', 'Shahid', 'House no 3', 'Johar Town','Lahore'),
(4,'Jamal', 'Aslam', 'House no 4', 'Kyaban town','Lahore'),
(5,'Rashid', 'Treen', 'House no 5', 'Gulberg 3','Lahore')
```

Insert Data Only in Specified Columns

- It is also possible to only insert data in specific columns.
- The following SQL statement will insert a new record, but only insert data in the “Persons", "City", and “FirstName“
- INSERT INTO Customers (PersonId, FirstName, City)
  VALUES (1, 'Cardinal', 'Stavanger');

SELECT Statement

- A typical SQL query has the form:

  select $A_1, A_2, ..., A_n$
  from $r_1, r_2, ..., r_m$
  where P

  - A_i represents an attribute
  - R_i represents a relation
  - P is a predicate.
- The result of an SQL query is a relation.

SELECT Statement

- The **select** clause lists the attributes desired in the result of a query
  - corresponds to the projection operation of the relational algebra
- Example: find the names of all instructors:

  select *name*

  from *instructor*
- NOTE: SQL names are case insensitive (i.e., you may use upper- or lower-case letters.)
  - E.g., *Name* ≡ *NAME* ≡ *name*
  - Some people use upper case wherever we use bold font.

SELECT Statement

- The SELECT statement is used to select data from a database.

SELECT *column1*, *column2, ...* **FROM *table_name***;

Or

SELECT * **FROM *table_name***;

* Represents the all columns of the table

SELECT Statement

- SQL allows duplicates in relations as well as in query results.
- To force the elimination of duplicates, insert the keyword **distinct** after select**.**
- Find the department names of all instructors, and remove duplicates

select distinct *dept_name*
from *instructor*

- The keyword **all** specifies that duplicates should not be removed.

select all *dept_name*
from *instructor*

dept_name
Comp. Sci.
Finance
Music
Physics
History
Physics
Comp. Sci.
History
Finance
Biology
Comp. Sci.
Elec. Eng.

SELECT Statement

- An asterisk in the select clause denotes “all attributes”

```
select *
from instructor
```

- An attribute can be a literal with no **from** clause

```
select '437'
```

  - Results is a table with one column and a single row with value “437”
  - Can give the column a name using:

```
select '437' as FOO
```

- An attribute can be a literal with **from** clause

```
select 'A'
from instructor
```

- Result is a table with one column and *N* rows (number of tuples in the *instructors* table), each row with value “A”

SELECT Statement

- The **select** clause can contain arithmetic expressions involving the operation, +, –, □, and /, and operating on constants or attributes of tuples.
  - The query:

    select *ID, name, salary/12*
    from *instructor*

    would return a relation that is the same as the *instructor* relation, except that the value of the attribute *salary* is divided by 12.
  - Can rename "s*alary/12"* using the **as** clause:

    select *ID, name, salary/12* **as** *monthly_salary*

Example

SELECT * **FROM** Persons;

Or,

Select PersonID,FirstName,LastName **From** Persons

(This will bring only selected columns(PersonID, FirstName, LastName) data from the table persons.

WHERE Clause

- The **where** clause specifies conditions that the result must satisfy
  - Corresponds to the selection predicate of the relational algebra.
- To find all instructors in Comp. Sci. dept

```
select name
from instructor
where dept_name = 'Comp. Sci.'
```

- SQL allows the use of the logical connectives **and, or,** and **not**
- The operands of the logical connectives can be expressions involving the comparison operators <, <=, >, >=, =, and <>.
- Comparisons can be applied to results of arithmetic expressions
- To find all instructors in Comp. Sci. dept with salary > 70000

```
select name
from instructor
where dept_name = 'Comp. Sci.'  and salary > 70000
```

WHERE Clause

- The where clause is used to filter the records. It is used to extract only those records that fulfill a specified condition.
- Syntax

  Select * from [Table] Where [CONDITION]
- SQL requires single quotes around text values (most database systems will also allow double quotes)

WHERE Clause

- The **where** clause specifies conditions that the result must satisfy
  - Corresponds to the selection predicate of the relational algebra.
- To find all instructors in Comp. Sci. dept

```
select name
from instructor
where dept_name = 'Comp. Sci.'
```

- SQL allows the use of the logical connectives **and, or,** and **not**
- The operands of the logical connectives can be expressions involving the comparison operators <, <=, >, >=, =, and <>.
- Comparisons can be applied to results of arithmetic expressions
- To find all instructors in Comp. Sci. dept with salary > 70000

```
select name
from instructor
where dept_name = 'Comp. Sci.' and salary > 70000
```

name
Katz
Brandt

WHERE Clause

Example:

Select * **from** Persons **where** city = ‘Lahore’

- For numeric value you have to write it without quotes

Select * **from** Persons **where** PersonID = 4

FROM clause

- The **from** clause lists the relations involved in the query
  - Corresponds to the Cartesian product operation of the relational algebra.
- Find the Cartesian product *instructor X teaches*

  select ∗
  from *instructor, teaches*

  - generates every possible instructor – teaches pair, with all attributes from both relations.
  - For common attributes (e.g., *ID*), the attributes in the resulting table are renamed using the relation name (e.g., *instructor.ID*)
- Cartesian product not very useful directly, but useful combined with where-clause condition (selection operation in relational algebra).

Operators

Operator	Description
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal. **Note:** In some versions of SQL this operator may be written as !=
BETWEEN	Between a certain range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column

Examples

- Find the names of all instructors who have taught some course and the course_id
  - **select** *name, course_id*
    from *instructor , teaches*
    where *instructor.ID = teaches.ID*

- Find the names of all instructors in the Art department who have taught some course and the course_id
  - **select** *name, course_id*
    from *instructor , teaches*
    where *instructor.ID = teaches.ID*
    and *instructor. dept_name* = 'Art'

name	*course_id*
Srinivasan	CS-101
Srinivasan	CS-315
Srinivasan	CS-347
Wu	FIN-201
Mozart	MU-199
Einstein	PHY-101
El Said	HIS-351
Katz	CS-101
Katz	CS-319
Crick	BIO-101
Crick	BIO-301
Brandt	CS-190
Brandt	CS-190
Brandt	CS-319
Kim	EE-181

AND, OR and NOT Operators

- The WHERE clause can be combined with AND, OR, and NOT operators.
- The AND and OR operators are used to filter records based on more than one condition:
- The AND operator displays a record if all the conditions separated by AND are TRUE.
- The OR operator displays a record if any of the conditions separated by OR is TRUE.
- The NOT operator displays a record if the condition(s) is NOT TRUE.

Syntax

- **SELECT** *column1*, *column2, ...*
  FROM *table_name*
  WHERE *condition1* **AND** *condition2* **AND** *condition3 ...*;
- **SELECT** *column1*, *column2, ...*
  FROM *table_name*
  WHERE *condition1* **OR** *condition2* **OR** *condition3 ...*;
- **SELECT** *column1*, *column2, ...*
  FROM *table_name*
  WHERE NOT *condition*;

Examples

- SELECT * FROM Customers
  WHERE Country = 'Germany' AND City = 'Berlin';

- SELECT * FROM Customers
  WHERE City = 'Berlin' OR City = 'Stuttgart';

- SELECT * FROM Customers
  WHERE NOT Country = 'Germany';

Activity

- Create a table **Employees** with the following records

Name	Sales_Rep_Number	Pay_Class	Rate
Ward	001	1	.05
Maxim	002	1	.05
Cane	003	1	.05
Beechum	004	2	.07
Collins	005	1	.05
Cannery	006	3	.09

Show all employees who have pay scale 2 and 3
Show employees who's Pay_Class is 1
Show all employees whos rate is above .05

NULL Values

- A field with a NULL value is a field with no value
- If a field in a table is optional, it is possible to insert a new record or update a record without adding a value to this field. Then, the field will be saved with a NULL value.
- It is not possible to test for NULL values with comparison operators, such as =, <, or <>.
- We will have to use the IS NULL and IS NOT NULL operators instead.

NULL Values

- Syntax

SELECT *column_names*
FROM *table_name*
WHERE *column_name* **IS NULL**;

SELECT *column_names*
FROM *table_name*
WHERE *column_name* **IS NOT NULL**;

Example

SELECT * **FROM** Persons **WHERE** City **IS NULL;**

SELECT * **FROM** Persons **WHERE** City **IS NOT NULL;**

Update Statement

- Update statement is used to update the existing record in the database.
- For example: A user address changes to it record should be updates in database as well.
- Update Syntax:

```
UPDATE table_name
SET column1 = value1,  column2 = value2,  . . .
WHERE condition;
```

Example:

```
UPDATE Persons
SET FirstName = 'Alfred', LastName = 'Schmidt', City= 'Frankfurt'
WHERE PersonID = 1;
```

DELETE Statement

- The DELETE statement is used to remove the existing record from the table.
- When we use **DELETE** statement with **WHERE** clause it deletes only that rows on which where condition satisfied.
- Without WHERE clause it will remove all data of the table.
- Syntax

```
DELETE FROM table_name WHERE [condition];
```

Example

- To delete all the data from table

```
DELETE FROM Persons
```

- To delete specific rows from the table

```
DELETE FROM Persons WHERE FirstName='Alfreds';
```

LIMIT Clause

- The LIMIT clause is used to specify the number of records to return.
- The LIMIT clause is useful on large tables with thousands of records. Returning a large number of records can impact performance.

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE condition
LIMIT number;
```

EXAMPLE

- `SELECT * FROM Persons LIMIT 3;`

ALIAS or The Rename Operation

- The SQL allows renaming relations and attributes using the **as** clause:

  old-name **as** *new-name*

- Find the names of all instructors who have a higher salary than some instructor in 'Comp. Sci'.
  - **select distinct** *T.name*
    from *instructor* **as** *T, instructor* **as** *S*
    where *T.salary > S.salary* **and** *S.dept_name = 'Comp. Sci.'*

- Keyword **as** is optional and may be omitted

  instructor **as** *T* ≡ *instructor T*

LIKE Operator

- The LIKE operator is used in a WHERE clause to search for a specified pattern in a column.

- There are two wildcards often used in conjunction with the LIKE operator:

  - The percent sign (%) represents zero, one, or multiple characters

  - The underscore sign (_) represents one, single character

- The percent sign and the underscore can also be used in combinations!

LIKE Operator

Symbol	Description	Example
%	Represents zero or more characters	bl% finds bl, black, blue, and blob
_	Represents a single character	h_t finds hot, hat, and hit

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that starts with "a"
WHERE CustomerName LIKE '%a'	Finds any values that ends with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a_%_%'	Finds any values that starts with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that starts with "a" and ends with "o"

SQL Constraints

- SQL constraints are used to specify the rules for the tables and also for data that is going to be inserted into table.
- In table level constraints can be specified
  - When the table is created (CREATE TABLE statement),
  - When changes need to done after creating table (ALTER TABLE statement).
- In column level constraints apply to a column.

SQL Constraints

- Constraints are used to limit the type of data that can go into a table.
- This ensures the accuracy and reliability of the data in the table.
- If there is any violation between the constraint and the data action, the action is aborted.

SQL Constraints

- NOT NULL - Ensures that a column cannot have a NULL value
- UNIQUE - Ensures that all values in a column are different
- PRIMARY KEY - A combination of a NOT NULL and UNIQUE. Uniquely identifies each row in a table
- FOREIGN KEY - Prevents actions that would destroy links between tables
- CHECK - Ensures that the values in a column satisfies a specific condition
- DEFAULT - Sets a default value for a column if no value is specified
- CREATE INDEX - Used to create

SQL NOT NULL Constraint

- The NOT NULL constraint enforces a column to NOT accept NULL values.
- By default, a column can hold NULL values.
- This enforces a field to always contain a value
- CREATE TABLE Persons (
  ID int **NOT NULL**,
  LastName varchar(255) **NOT NULL**,
  FirstName varchar(255) **NOT NULL**,
  Age int
  );

SQL NOT NULL Constraint

- SQL
  - ALTER TABLE Persons
    ALTER COLUMN Age int NOT NULL;
- My SQL
  - ALTER TABLE Persons
    MODIFY Age int NOT NULL;

SQL PRIMARY KEY Constraint

- The **PRIMARY KEY** constraint uniquely identifies each record in a table.
- Primary keys must contain UNIQUE values, and cannot contain NULL values.
- A table can have only ONE primary key; and in the table, this primary key can consist of single or multiple columns (fields).

SQL PRIMARY KEY Constraint

Example

- CREATE TABLE Persons (
  ID int NOT NULL PRIMARY KEY,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Age int
  );

Example (MYSQL)

- CREATE TABLE Persons (
  ID int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Age int,
  PRIMARY KEY (ID)
  );

SQL PRIMARY KEY Constraint

- To allow naming of a PRIMARY KEY constraint, and for defining a PRIMARY KEY constraint on multiple columns, use the following SQL syntax:
  - CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    CONSTRAINT PK_Person PRIMARY KEY (ID,LastName)
    );
- In the example above there is only ONE PRIMARY KEY (PK_Person). However, the VALUE of the primary key is made up of TWO COLUMNS (ID + LastName).

SQL PRIMARY KEY on ALTER TABLE

- To create a PRIMARY KEY constraint on the "ID" column when the table is already created, use the following SQL:
  - ALTER TABLE Persons
    ADD PRIMARY KEY (ID);

SQL PRIMARY KEY on ALTER TABLE

- To allow naming of a PRIMARY KEY constraint, and for defining a PRIMARY KEY constraint on multiple columns, use the following SQL syntax:
  - ALTER TABLE Persons
    ADD CONSTRAINT PK_Person PRIMARY KEY (ID,LastName);

DROP a PRIMARY KEY Constraint

- SQL:
  - ALTER TABLE Persons
    DROP CONSTRAINT PK_Person;
- MySQL
  - ALTER TABLE Persons
    DROP PRIMARY KEY;

SQL FOREIGN KEY Constraint

- The FOREIGN KEY constraint is used to prevent actions that would destroy links between tables.
- A FOREIGN KEY is a field (or collection of fields) in one table, that refers to the PRIMARY KEY in another table.
- The table with the foreign key is called the child table, and the table with the primary key is called the referenced or parent table.

SQL FOREIGN KEY Constraint

- SQL
  - CREATE TABLE Orders (
    OrderID int NOT NULL PRIMARY KEY,
    OrderNumber int NOT NULL,
    PersonID int FOREIGN KEY REFERENCES Persons(PersonID)
    );
- MYSQL
  - CREATE TABLE Orders (
    OrderID int NOT NULL,
    OrderNumber int NOT NULL,
    PersonID int,
    PRIMARY KEY (OrderID),
    FOREIGN KEY (PersonID) REFERENCES Persons(PersonID)
    );

SQL FOREIGN KEY Constraint

- To allow naming of a FOREIGN KEY constraint, and for defining a FOREIGN KEY constraint on multiple columns, use the following SQL syntax:
- **CREATE TABLE** Orders (
  OrderID int NOT NULL,
  OrderNumber int NOT NULL,
  PersonID int,
  PRIMARY KEY (OrderID),
  CONSTRAINT FK_PersonOrder FOREIGN KEY (PersonID)
  REFERENCES Persons(PersonID)
  );

SQL FOREIGN KEY on ALTER TABLE

- **Example**
  - ALTER TABLE Orders
    ADD FOREIGN KEY (PersonID) REFERENCES Persons(PersonID);
- To allow naming of a FOREIGN KEY constraint, and for defining a FOREIGN KEY constraint on multiple columns
  - ALTER TABLE Orders
    ADD CONSTRAINT FK_PersonOrder
    FOREIGN KEY (PersonID) REFERENCES Persons(PersonID);

DROP a FOREIGN KEY Constraint

- **SQL**
  - ALTER TABLE Orders
    DROP CONSTRAINT FK_PersonOrder;
- **MySQL**
  - ALTER TABLE Orders
    DROP FOREIGN KEY FK_PersonOrder;

SQL UNIQUE Constraint

- The UNIQUE constraint ensures that all values in a column are different.
- Both the UNIQUE and PRIMARY KEY constraints provide a guarantee for uniqueness for a column or set of columns.
- A PRIMARY KEY constraint automatically has a UNIQUE constraint.
- However, you can have many UNIQUE constraints per table, but only one PRIMARY KEY constraint per table.

SQL UNIQUE Constraint

- Example
- SQL
  - CREATE TABLE Persons (
    ID int NOT NULL UNIQUE,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int
    );
- MySQL
  - CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    UNIQUE (ID)
    );

SQL UNIQUE Constraint

- To name a UNIQUE constraint, and to define a UNIQUE constraint on multiple columns, use the following SQL syntax:
  - CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    CONSTRAINT UC_Person UNIQUE (ID,LastName)
    );

SQL CHECK Constraint

- The CHECK constraint is used to limit the value range that can be placed in a column.
- If you define a CHECK constraint on a column it will allow only certain values for this column.
- If you define a CHECK constraint on a table it can limit the values in certain columns based on values in other columns in the row.

SQL CHECK on CREATE TABLE

- The following SQL creates a CHECK constraint on the "Age" column when the "Persons" table is created. The CHECK constraint ensures that the age of a person must be 18, or older:

Example

- MySQL
  - CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    CHECK (Age>=18)
    );
- SQL
  - CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int CHECK (Age>=18)
    );

SQL CHECK on Multiple Column

- To allow naming of a CHECK constraint, and for defining a CHECK constraint on multiple columns:
  - CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    City varchar(255),
    CONSTRAINT CHK_Person CHECK (Age>=18 AND City='Sandnes')
    );

SQL CHECK on ALTER TABLE

- Example
  - ALTER TABLE Persons
    ADD CHECK (Age>=18);
- Multi-columns
  - ALTER TABLE Persons
    ADD CONSTRAINT CHK_PersonAge CHECK (Age>=18 AND City='Sandnes');

DROP a CHECK Constraint

- Example:
- SQL
  - ALTER TABLE Persons
    DROP CONSTRAINT CHK_PersonAge;
- MySQL
  - ALTER TABLE Persons
    DROP CHECK CHK_PersonAge;

SQL DEFAULT Constraint

- The DEFAULT constraint is used to set a default value for a column.
- The default value will be added to all new records, if no other value is specified.
  - CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    City varchar(255) DEFAULT 'Sandnes'
    );

SQL DEFAULT Constraint

- CREATE TABLE Orders (
    ID int NOT NULL,
    OrderNumber int NOT NULL,
    OrderDate date DEFAULT GETDATE()
  );

SQL DEFAULT on ALTER TABLE

- SQL
  - ALTER TABLE Persons
    ADD CONSTRAINT df_City
    DEFAULT 'Sandnes' FOR City;
- MySQL
  - ALTER TABLE Persons
    ALTER City SET DEFAULT 'Sandnes';

DROP a DEFAULT Constraint

- SQL
  - ALTER TABLE Persons
    ALTER COLUMN City DROP DEFAULT;
- MySQL
  - ALTER TABLE Persons
    ALTER City DROP DEFAULT;

SQL CREATE INDEX Statement

- The CREATE INDEX statement is used to create indexes in tables.
- Indexes are used to retrieve data from the database more quickly than otherwise. The users cannot see the indexes, they are just used to speed up searches/queries.

CREATE INDEX

- Creates an index on a table. Duplicate values are allowed:
  - CREATE INDEX *index_name*
    ON *table_name* (*column1*, *column2*, ...);
- Creates a unique index on a table. Duplicate values are not allowed:
  - CREATE UNIQUE INDEX *index_name*
    ON *table_name* (*column1*, *column2*, ...);

Example

- creates an index named "idx_lastname" on the "LastName" column in the "Persons" table
  - CREATE INDEX idx_lastname
    ON Persons (LastName);
- If you want to create an index on a combination of columns, you can list the column names within the parentheses, separated by commas:
  - CREATE INDEX idx_pname
    ON Persons (LastName, FirstName);

UNION Operator

- The UNION operator is used to combine the result-set of two or more SELECT statements.
  - Every SELECT statement within UNION must have the same number of columns
  - The columns must also have similar data types
  - The columns in every SELECT statement must also be in the same order
- Syntax
  - SELECT *column_name(s)* FROM *table1*
    UNION
    SELECT *column_name(s)* FROM *table2*;

UNION ALL Syntax

- The UNION operator selects only distinct values by default. To allow duplicate values, use UNION ALL:
- Sytanx
  - SELECT *column_name(s)* FROM *table1*
    UNION ALL
    SELECT *column_name(s)* FROM *table2*;

Example

- SELECT City FROM Customers
  UNION
  SELECT City FROM Suppliers
  ORDER BY City;
- SELECT City FROM Customers
  UNION ALL
  SELECT City FROM Suppliers
  ORDER BY City;

- SELECT City, Country FROM Customers
  WHERE Country='Germany'
  UNION
  SELECT City, Country FROM Suppliers
  WHERE Country='Germany'
  ORDER BY City;

INTERSECT operator

- The SQL **INTERSECT** clause/operator is used to combine two SELECT statements, but returns rows only from the first SELECT statement that are identical to a row in the second SELECT statement.
- This means INTERSECT returns only common rows returned by the two SELECT statements.
- Just as with the UNION operator, the same rules apply when using the INTERSECT operator.
- MySQL does not support the INTERSECT operator.

Syntax

SELECT column1 [, column2]
FROM table1 [, table2]
[WHERE condition]

INTERSECT

SELECT column1 [, column2]
FROM table1 [, table2]
[WHERE condition]

Example

Select Name from Buyers

Intersect

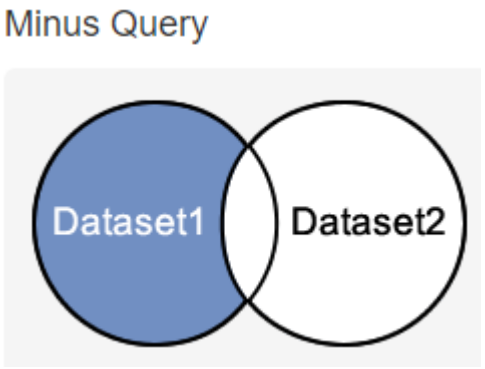
Select Name from Suppliers

MINUS Operator

- The SQL MINUS operator is used to return all rows in the first SELECT statement that are not returned by the second SELECT statement.
- Each SELECT statement will define a dataset.
- The MINUS operator will retrieve all records from the first dataset and then remove from the results all records from the second dataset.
- Each SELECT statement within the MINUS query must have the same number of fields in the result sets with similar data types.
- he MINUS operator is not supported in all SQL databases. It can used in databases such as Oracle.
- For databases such as SQL Server, PostgreSQL, and SQLite, use the **EXCEPT operator** to perform this type of query.

MINUS Operator

- The MINUS query will return the records in the blue shaded area. These are the records that exist in Dataset1 and not in Dataset2.



Syntax

```
SELECT expression1, expression2, ... expression_n
FROM tables
[WHERE conditions]
MINUS
SELECT expression1, expression2, ... expression_n
FROM tables
[WHERE conditions];
```

Example

```
SELECT supplier_id
FROM suppliers
MINUS
SELECT supplier_id
FROM orders;
```

EXCEPT Operator

- This SQL tutorial explains how to use the SQL **EXCEPT operator** with syntax and examples.
- The SQL EXCEPT operator is used to return all rows in the first SELECT statement that are not returned by the second SELECT statement.
- Each SELECT statement will define a dataset.
- The EXCEPT operator will retrieve all records from the first dataset and then remove from the results all records from the second dataset.

EXCEPT Operator

- Syntax

```
SELECT expression1, expression2, ... expression_n
FROM tables
[WHERE conditions]
EXCEPT
SELECT expression1, expression2, ... expression_n
FROM tables
[WHERE conditions];
```

EXCEPT Operator

- This EXCEPT operator example returns all *product_id* values that are in the *products* table and not in the *inventory* table.

```
SELECT product_id
FROM products
EXCEPT
SELECT product_id
FROM inventory;
```

SQL IN Operator

- The IN operator allows you to specify multiple values in a WHERE clause.
- The IN operator is a shorthand for multiple OR conditions.
  - SELECT *column_name(s)*
    FROM *table_name*
    WHERE *column_name* IN (*value1*, *value2*, ...);
- Example
  - SELECT * FROM Customers
    WHERE Country IN ('Germany', 'France', 'UK');

BETWEEN Operator

- The BETWEEN operator selects values within a given range.
- The values can be numbers, text, or dates.
- The BETWEEN operator is inclusive: begin and end values are included.
  - SELECT *column_name(s)*
    FROM *table_name*
    WHERE *column_name* BETWEEN *value1* AND *value2;*

Example

- Example
  - SELECT * FROM Products
    WHERE Price BETWEEN 10 AND 20;
- NOT Between
  - SELECT * FROM Products
    WHERE Price NOT BETWEEN 10 AND 20;

BETWEEN with IN

- Example
  - SELECT * FROM Products
    WHERE Price BETWEEN 10 AND 20
    AND CategoryID NOT IN (1,2,3);
- Between in Text
  - SELECT * FROM Products
    WHERE ProductName BETWEEN 'Carnarvon Tigers' AND 'Mozzarella di Giovanni'
    ORDER BY ProductName;

SELECT DISTINCT Statement

- The SELECT DISTINCT statement is used to return only distinct (different) values.
- Inside a table, a column often contains many duplicate values; and sometimes you only want to list the different (distinct) values.
  - SELECT DISTINCT *column1*, *column2, ...*
    FROM *table_name*;

Example

- SELECT Country FROM Customers;